

SERVICIO NACIONAL DE APRENDIZAJE - SENA

EVIDENCIA GA7-220501096-AA3-EV01

CODIFICACIÓN DE MÓDULOS DEL SOFTWARE STAND-ALONE, WEB Y MÓVIL

Módulo de Inventario con el framework Spring Boot - Taller Lizardo Car

Aprendiz:

Alejandro Cardoso Parra

Programa de Formación:

ANÁLISIS Y DESARROLLO DE SOFTWARE

Ficha:

3186674

Instructor:

ANDRÉS FELIPE PARRA MARTINEZ

2026

1. Datos generales

Campo	Descripción
Programa de formación	Análisis y Desarrollo de Software
Proyecto formativo	Construcción de software integrador de tecnologías orientadas a servicios.
Fase del proyecto	Ejecución
Resultado de aprendizaje	220501096-04 - Codificar el software de acuerdo con el diseño establecido.
Actividad de aprendizaje	GA7-220501096-AA3 - Codificar los módulos del software stand-alone, web y móvil, de acuerdo con el diseño.
Evidencia de desempeño	Codificación de módulos del software stand-alone, web y móvil. GA7-220501096-AA3-EV01.
Criterio de evaluación	Codifica los módulos del software stand-alone, web y móvil, de acuerdo con las especificaciones del diseño y el estándar de codificación.

2. Introducción

Esta evidencia continúa la codificación del módulo de Inventario del sistema "Taller Lizardo Car", incorporando esta vez un framework de desarrollo: Spring Boot. A diferencia de las evidencias AA2-EV01 (consola con JDBC puro) y AA2-EV02 (web con Servlets y JSP puros), aquí el acceso a datos ya no se escribe a mano con sentencias SQL, sino que se apoya en Spring Data JPA, la herramienta de almacenamiento de datos que integra el propio framework.

El módulo conserva el mismo dominio (productos del inventario) y la misma base de datos utilizada en las evidencias anteriores, pero reorganiza el código en una arquitectura por capas típica de Spring (controlador, servicio, repositorio, entidad), aplicando el estándar de codificación ya definido para el proyecto.

3. Objetivo

3.1 Objetivo general

Codificar el módulo de Inventario del software "Taller Lizardo Car" utilizando el framework Spring Boot, integrando una herramienta de almacenamiento de datos, aplicando un estándar de codificación con comentarios, y utilizando herramientas de versionamiento.

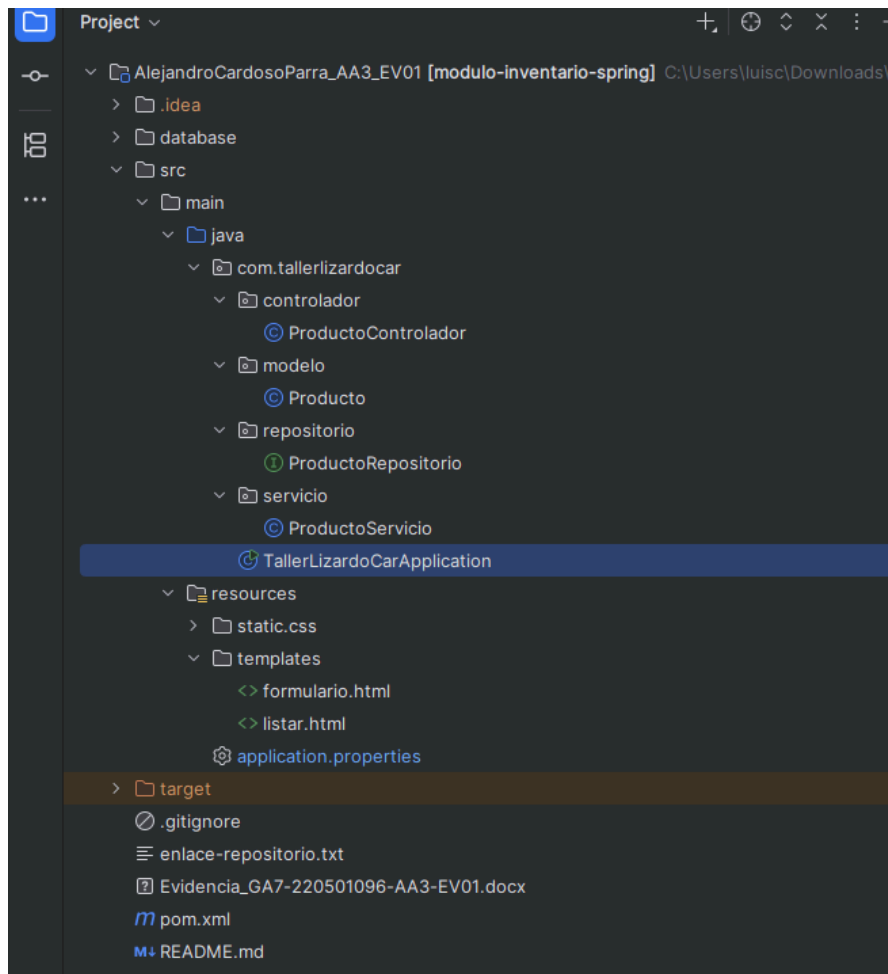
3.2 Objetivos específicos

- Seleccionar y aplicar Spring Boot como framework de codificación del módulo.
- Integrar Spring Data JPA como herramienta de almacenamiento de datos, reemplazando el JDBC manual.
- Aplicar el estándar de codificación del proyecto y documentar el código con comentarios tipo Javadoc.
- Registrar el desarrollo del módulo mediante un repositorio Git con commits organizados por capa.

4. Artefactos de referencia y framework seleccionado

- Diseño de datos y caso de uso "Gestionar inventario": los mismos definidos desde la evidencia AA3-EV02 (de la actividad AA... de diseño) y ya codificados en las evidencias AA2-EV01 y AA2-EV02.
- Framework seleccionado: Spring Boot, por ser el framework de desarrollo web en Java más usado en la industria y una evolución natural de los Servlets puros ya trabajados.
- Herramienta de almacenamiento de datos integrada: Spring Data JPA (con Hibernate), que genera automáticamente las operaciones CRUD a partir de una interfaz de repositorio.
- Estándar de codificación: se mantiene el definido en la evidencia AA1-EV02 (camelCase, PascalCase, paquetes en minúsculas), ahora aplicado también a las anotaciones y a la arquitectura por capas de Spring.

5. Estructura del módulo codificado



6. Framework aplicado: Spring Boot

6.1 Arquitectura por capas

Capa	Clase	Responsabilidad
Entidad (modelo)	Producto	Mapea la clase a la tabla productos mediante anotaciones JPA (@Entity, @Table, @Id).
Repositorio	ProductoRepositorio	Extiende JpaRepository; Spring genera el CRUD automáticamente.
Servicio	ProductoServicio	Contiene la lógica de negocio e intermedia entre el controlador y el repositorio.
Controlador	ProductoControlador	Expone las rutas web (GET y POST) y decide qué vista Thymeleaf renderizar.

6.2 Integración de la herramienta de almacenamiento de datos

Spring Data JPA es la herramienta de almacenamiento de datos integrada en este módulo. A diferencia del JDBC manual de las evidencias anteriores (donde cada operación CRUD requería escribir su propia sentencia SQL con PreparedStatement), aquí basta con declarar una interfaz que extienda JpaRepository:

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure with folders for 'src', 'main', 'resources', 'static', 'templates', and 'target'. The 'main' folder contains 'com.tallerlizardocar' with sub-packages 'controlador', 'modelo', 'repositorio', 'servicio', and 'TallerLizardoCarApplication'.
- Code Editor:** Displays the 'ProductoRepositorio.java' file. The code defines a package, imports, and a class that extends 'JpaRepository'. It includes a comment in Spanish explaining the purpose of the repository and the use of Spring Data JPA. The class definition is:


```

package com.tallerlizardocar.repositorio;

import org.springframework.data.jpa.repository.JpaRepository;

/**
 * Repositorio de acceso a datos para la entidad Producto.
 *
 * Al extender JpaRepository, Spring Data JPA genera automáticamente
 * en tiempo de ejecución la implementación de las operaciones CRUD
 * básicas (save, findById, findByIds, deleteById, etc.) sin necesidad de
 * escribir una sola sentencia SQL ni una clase de implementación.
 * esto es la herramienta de almacenamiento de datos que exige la
 * evidencia, integrada mediante el framework Spring.
 */
public class ProductoRepositorio extends JpaRepository<Producto, Integer> {
}

```
- Run Console:** Shows the execution logs of the application. The logs indicate that the application started successfully on port 8080. The logs are:


```

2020-09-13T19:42:01.602-05:00 WARN 13388 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. E
2020-09-13T19:42:01.961-05:00 INFO 13388 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2020-09-13T19:42:01.968-05:00 INFO 13388 --- [main] c.t.TallerLizardoCarApplication : Started TallerLizardoCarApplication in 4.523 seconds (process running for 5.174)
2020-09-13T20:58:58.022-05:00 WARN 13388 --- [t-1 housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=50m27550ms649us).
2020-09-13T21:44:48.925-05:00 WARN 13388 --- [t-1 housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=7m20s333ms826us).

```

6.3 Rutas GET y POST

Ruta	Método	Qué hace
/productos	GET	Lista todos los productos (listar.html).
/productos/nuevo	GET	Muestra el formulario vacío.
/productos/editar/{id}	GET	Muestra el formulario precargado con los datos del producto.
/productos/guardar	POST	Recibe el formulario e inserta o actualiza según si el id viene vacío.
/productos/eliminar/{id}	GET	Elimina el producto y redirige al listado.

7. Uso de herramientas de versionamiento

El módulo se entrega con un repositorio Git local, con commits organizados por cada capa de la arquitectura:



```

Terminal Local x v
Windows PowerShell
Copyright (c) Microsoft Corporation. All rights reserved.

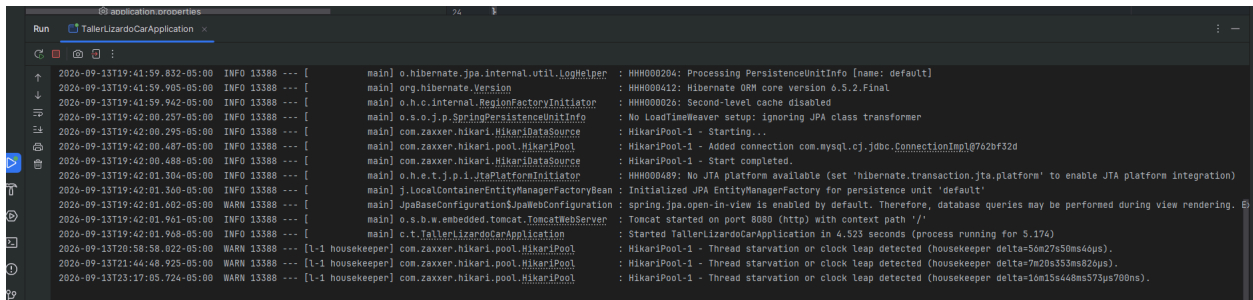
PS C:\Users\luisc\Downloads\AlejandroCardosoParra_AA3_EV01> git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/main/java/com/tallerlizardocar/repositorio/ProductoRepositorio.java
        modified:   src/main/resources/application.properties

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisc\Downloads\AlejandroCardosoParra_AA3_EV01> git add
warning: in the working copy of 'src/main/java/com/tallerlizardocar/repositorio/ProductoRepositorio.java', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/resources/application.properties', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\luisc\Downloads\AlejandroCardosoParra_AA3_EV01> git commit -m "create modules app"
[main 779af3e] create modules app
 2 files changed, 3 insertions(+), 15 deletions(-)
PS C:\Users\luisc\Downloads\AlejandroCardosoParra_AA3_EV01>
  
```

Figura 1. Commits

8. Capturas de la ejecución y prueba del módulo

8.1 Arranque de la aplicación



```

Run
TallerLizardoCarApplication
2026-09-13T19:41:59.832-05:00 INFO 13388 --- [main] o.h.i.p.a.i.u.l.l.o.g.g.e.l.p.e.r : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-09-13T19:41:59.905-05:00 INFO 13388 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM core version 6.5.2.Final
2026-09-13T19:41:59.942-05:00 INFO 13388 --- [main] o.h.c.i.n.t.e.r.n.a.l.R.e.g.i.o.n.F.a.c.t.o.r.y.I.n.i.t.i.a.t.o.r : HHH000026: Second-level cache disabled
2026-09-13T19:42:00.257-05:00 INFO 13388 --- [main] o.s.o.j.p.S.p.r.i.n.g.P.e.r.s.i.s.t.e.n.c.e.U.n.i.t.i.n.f.o : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-09-13T19:42:00.295-05:00 INFO 13388 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-09-13T19:42:00.487-05:00 INFO 13388 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@762bf32d
2026-09-13T19:42:00.488-05:00 INFO 13388 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-09-13T19:42:01.304-05:00 INFO 13388 --- [main] o.h.e.t.j.p.i.j.t.a.p.l.a.t.f.o.r.m.i.n.i.t.i.a.t.o.r : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-09-13T19:42:01.305-05:00 INFO 13388 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-09-13T19:42:01.402-05:00 WARN 13388 --- [main] jp.h.a.s.c.o.n.f.i.g.u.r.a.t.i.o.n.s.p.y.m.a.s.c.o.n.f.i.g.u.r.a.t.i.o.n : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering.
2026-09-13T19:42:01.961-05:00 INFO 13388 --- [main] o.s.b.w.e.m.b.e.d.d.e.d.t.o.m.c.a.t.T.o.m.c.a.t.W.e.b.s.e.r.v.e.r : Tomcat started on port 8080 (http) with context path '/'
2026-09-13T19:42:01.968-05:00 INFO 13388 --- [main] c.t.t.a.l.l.e.r.l.i.z.a.r.d.o.c.a.r.A.p.p.l.i.c.a.t.i.o.n : Started TallerLizardoCarApplication in 4.323 seconds (process running for 5.174)
2026-09-13T20:58:58.022-05:00 WARN 13388 --- [l-1 housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=50m27450ms449us).
2026-09-13T21:44:48.925-05:00 WARN 13388 --- [l-1 housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=7m20s353ms826us).
2026-09-13T23:17:05.724-05:00 WARN 13388 --- [l-1 housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=10m15s448ms573us700ms).
  
```

Figura 3. Consola mostrando el arranque de Spring Boot (Tomcat started on port 8080).

8.2 Listado de productos (GET)

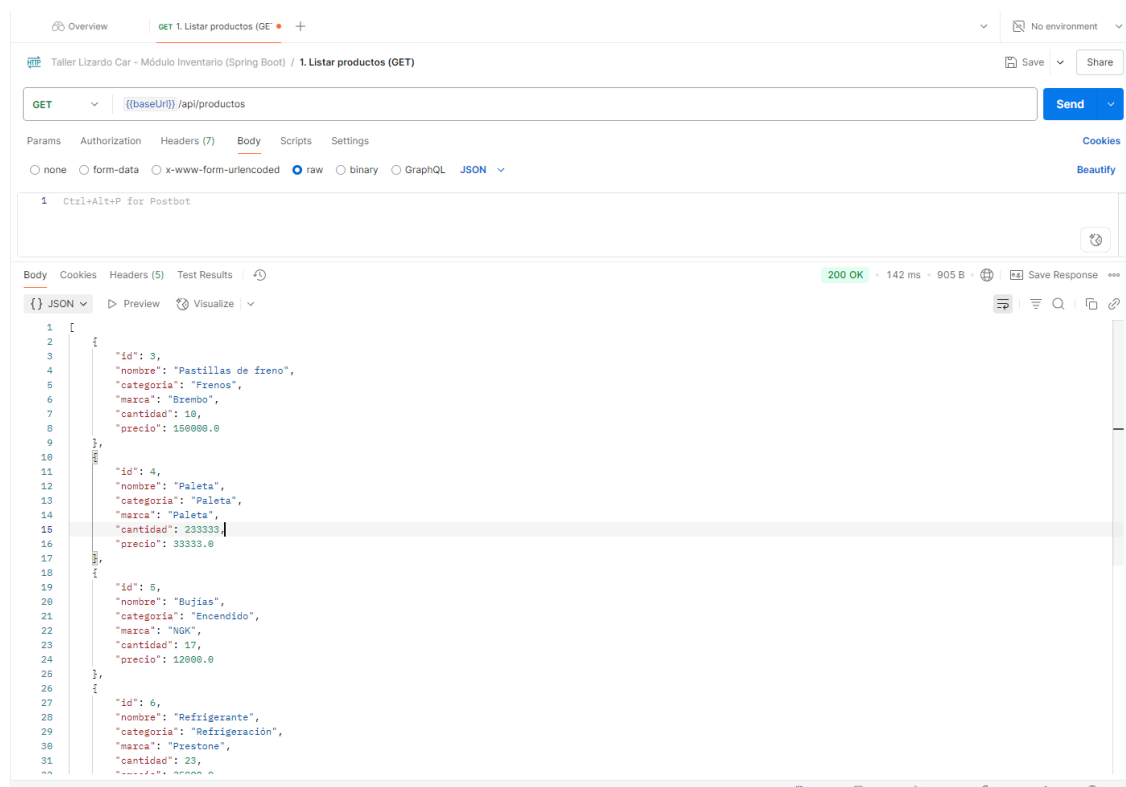


Figura 4. Página listar.html mostrando los productos del inventario.

9.3 Creación de un producto (POST)

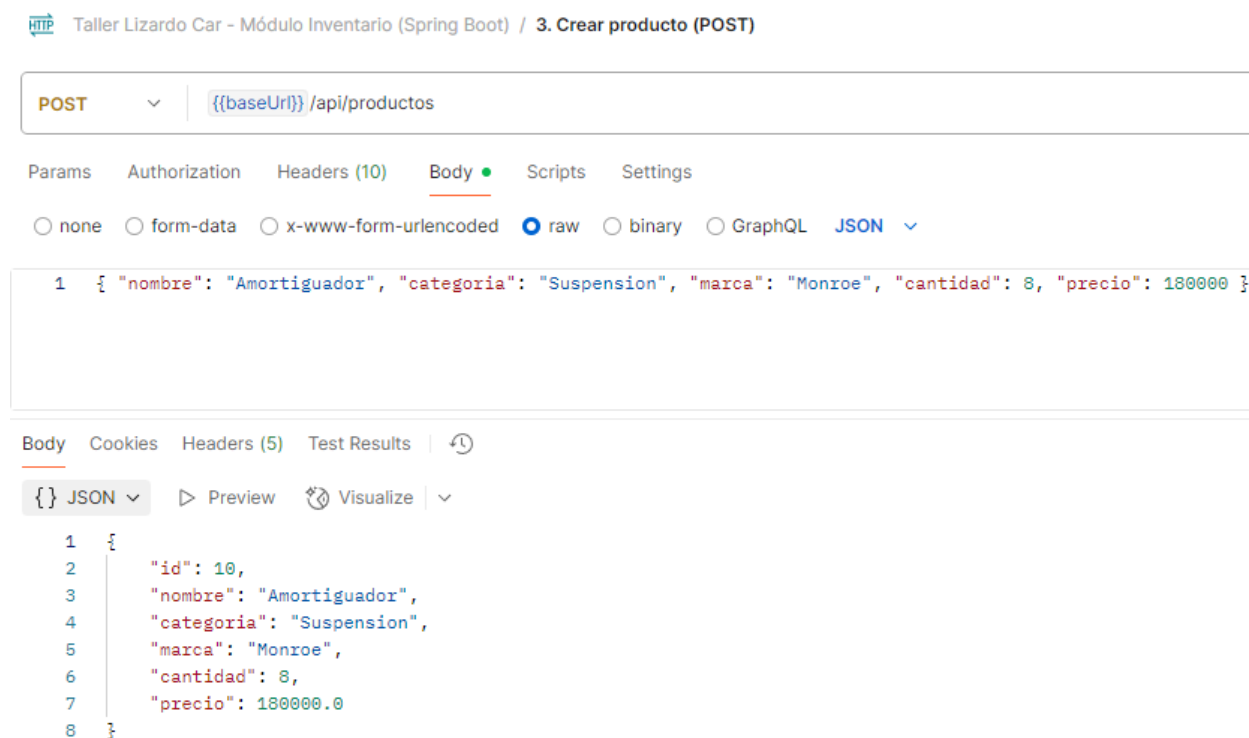


Figura 5. Listado actualizado tras crear un nuevo producto desde el formulario.

9.4 Edición y actualización (GET + POST)

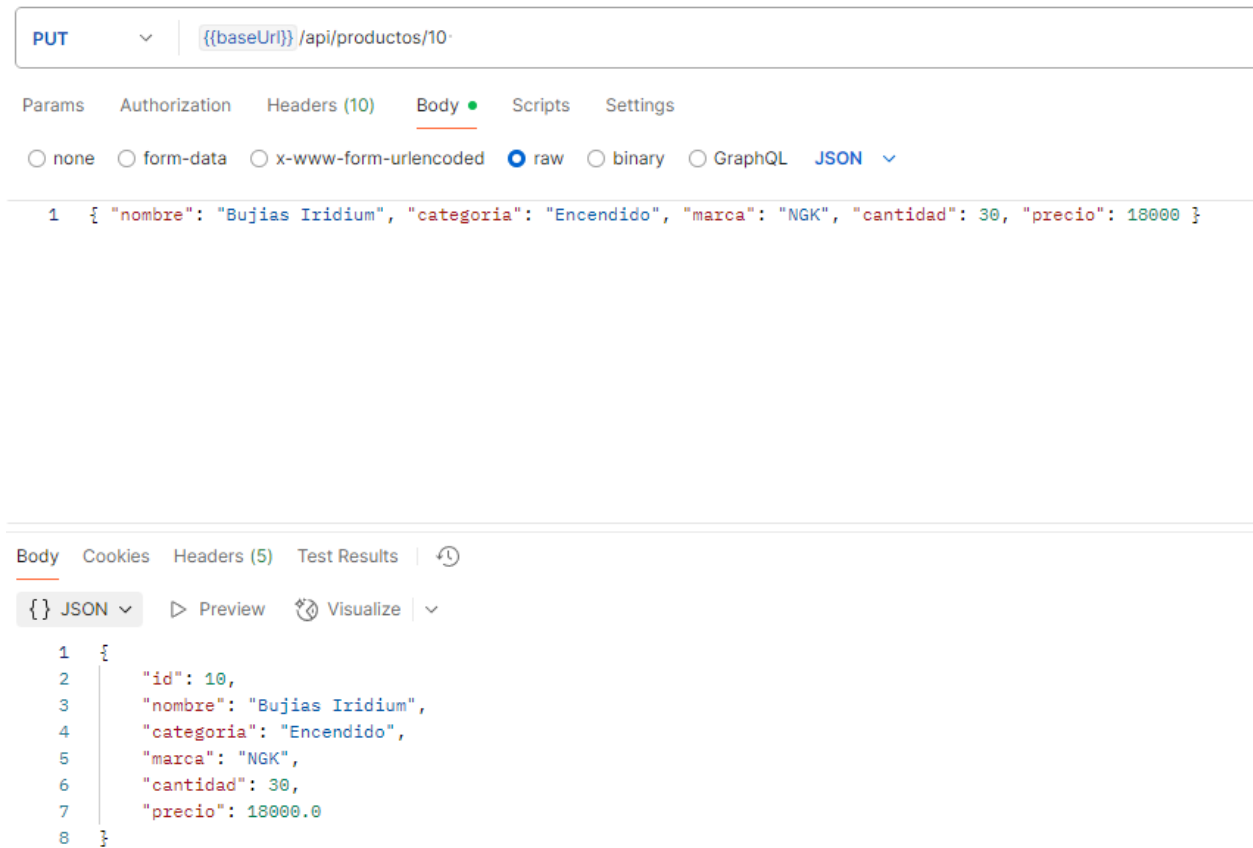
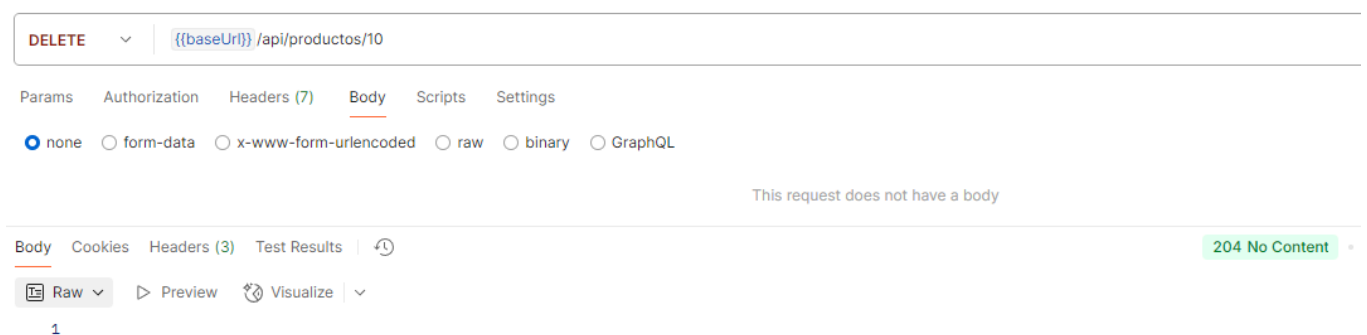


Figura 6. Formulario precargado y listado reflejando los cambios tras actualizar.

9.5 Eliminación de un producto

Figura 7. Listado sin el producto eliminado.



10. Conclusiones

- La migración de JDBC manual a Spring Data JPA redujo notablemente el código necesario para el CRUD, sin perder el control sobre la lógica de negocio, que quedó aislada en la capa de servicio.
- La arquitectura por capas de Spring (entidad, repositorio, servicio, controlador) ordena el proyecto de una forma más clara que el switch manual sobre un parámetro de acción usado en la versión con Servlets puros.
- Aplicar un framework no elimina la necesidad de un estándar de codificación propio: los nombres de clases, métodos y variables siguen las mismas convenciones definidas desde la evidencia AA1-EV02.
- Queda pendiente, como responsabilidad del aprendiz, compilar y ejecutar este módulo en un entorno con acceso completo a Maven Central, documentando el resultado con las capturas de la sección 9.